OS ORANGE

Name – Aaron Sijo

SRN – PES2UG24CS012

PHASE – 1

```
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

```
aaron@AaronSijo:~/PES2UG24CS012-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
```

PHASE – 2

```
aaron@AaronSijo:~/PES2UG24CS012-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization
```

```
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
```

```
6a1af8e11ec4f476
00000000: 626c 6f62 2031 3800 4475 706c 6963 6174  blob 18.Duplicat
00000010: 6520 636f 6e74 656e 740a                 e content.
```

PHASE – 3

```
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  .gitignore
  untracked:  commit.c
  untracked:  commit.h
  untracked:  index.c
  untracked:  index.h
  untracked:  Makefile
  untracked:  object.c
  untracked:  pes.c
  untracked:  pes.h
  untracked:  README.md
  untracked:  test_objects.c
  untracked:  test_sequence.sh
  untracked:  test_tree.c
  untracked:  tree.c
  untracked:  tree.h
```

```
100777 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776334047 6 file1.txt
100777 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776334052 6 file2.txt
```

PHASE – 4

```
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/09/755aee07797961c22b9701dd788ae2e21ca8eac86e0ee1221b000055b8a6b1
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/a4/34f8b1bf184edcfb0a2eb35fd6d405ae02fa246b7205d26ad9f5c087266025
.pes/objects/d6/dfaf358f4da1a003e5a16a953e496962f93ecb6809f767d8bbfc3a926a0b58
.pes/objects/e4/0ad773cb975ed3e06d46f1ae6642c290a49f9274a8fd028cfef5bc4723ed09
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/ea/abeb710475acf500f6636b1507e573174a5a47d09906b1917edc5fe4fb17fb
.pes/objects/f4/7964017e8424936d155636939bbf9d8e30253597a5536aea0af97ccb1682b7
.pes/refs/heads/main
```

```
09755aee07797961c22b9701dd788ae2e21ca8eac86e0ee1221b000055b8a6b1
```

FINAL

```
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 019023785133... Initial commit
```

```
    Initial committial comQ


--- Second Commit ---
Committed: 1ced168325f4... Update file.txt

--- Third Commit ---
Committed: b1e5e51132f1... Add farewell
```

```
--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
b1e5e51132f145b27f5dceb158989c04a422c9830b164fdb83f0ddb97222fe79

--- Object Store ---
Objects created:
10
.pes/objects/01/9023785133e18657daf11fe363962f6a06a6399c2117dabc36fbeda6546582
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b5253611147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/1c/ed168325f4077ec6230f24cb75abc6bb9cec5a068d0637dc7e45f7a8ac73c2
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/b1/e5e51132f145b27f5dceb158989c04a422c9830b164fdb83f0ddb97222fe79
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
```

PHASE 5 and 6

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

To implement pes checkout , the file .pes/refs/heads/ is read to get the commit hash, and .pes/HEAD is updated to point to that branch. The working directory must then be updated to match the tree of that commit by recreating all files and directories from the object store. The index must also be updated accordingly. This operation is complex because it involves safely overwriting files, handling directory structures, and ensuring uncommitted changes are not lost.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

For each file in the index, compute the current hash from the working directory and compare it with the stored hash in the index. If they differ, the file has uncommitted changes. Then compare the index version with the version in the target branch’s tree from the object store. If both differ, a conflict exists. In such cases, checkout must be refused to prevent overwriting user changes

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

In a detached HEAD state, new commits are created normally but are not referenced by any branch. HEAD moves forward, but no branch points to these commits, making them potentially unreachable later. A user can recover them by creating a new branch pointing to the current HEAD or by manually saving the commit hash in a branch reference.

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Start from all branch heads and add their commit hashes to a set of reachable objects. Traverse each commit to collect its parent and tree, and traverse trees to collect all referenced blobs and subtrees. Continue until all reachable objects are found. Then delete any objects in .pes/objects not present in this set. A hash set should be used for efficient lookup. For 100,000 commits, the total objects visited could be around 300,000 to 500,000 including commits, trees, and blobs.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

It is dangerous because a commit operation may create new objects that are not yet referenced by any branch. If garbage collection runs at the same time, it may consider these objects unreachable and delete them. This creates a race condition where a commit refers to objects that no longer exist. Git avoids this by using locks and by only deleting

objects that are safely unreachable for a certain period, ensuring active operations are not affected.